

计算机体系结构

L06-1 多处理器

计算机体系结构

背景知识 (自学) ③

存储层次 ④

单处理器 (指令级并行) ④

多处理器 (线程级并行)

多处理分类

松耦合多处理器

紧耦合多处理器

对称多处理器

NUMA

Cache 一致性 (Coherence) ②

同步 (Synchronization) ②

内存一致性 (Consistency) ③

互连网络 (Interconnect) ②

加速器 (数据并行) ③

多处理器分类

为何需要多处理器(Multiprocessors)?

- **提升性能 (任务执行时间或者吞吐量)**
- **减小功耗:**
 - 频率为 $F/4$ 的 $4N$ 个核比频率为 F 的 N 个核功耗小
- **减少硬件复杂度，提升可扩展性**
- **提高可靠性：空间上的冗余执行**

多处理器分类

□ 松耦合多处理器

- ✓ 没有共享的全局内存地址空间（进程）
- ✓ 多个处理器通过网络连接
- ✓ 通常通过消息传递进行编程
 - 调用send, receive等函数实现通信

□ 紧耦合多处理器

- ✓ 共享的全局内存地址空间
- ✓ 编程模型类似于单处理器（即多任务单处理器）
- ✓ 线程通过共享变量（内存）进行协作，而对共享数据的操作需要同步

松耦合 vs 紧耦合

```
for (i = 0; i < 10000; i++)  
    sum = sum + A[i];
```

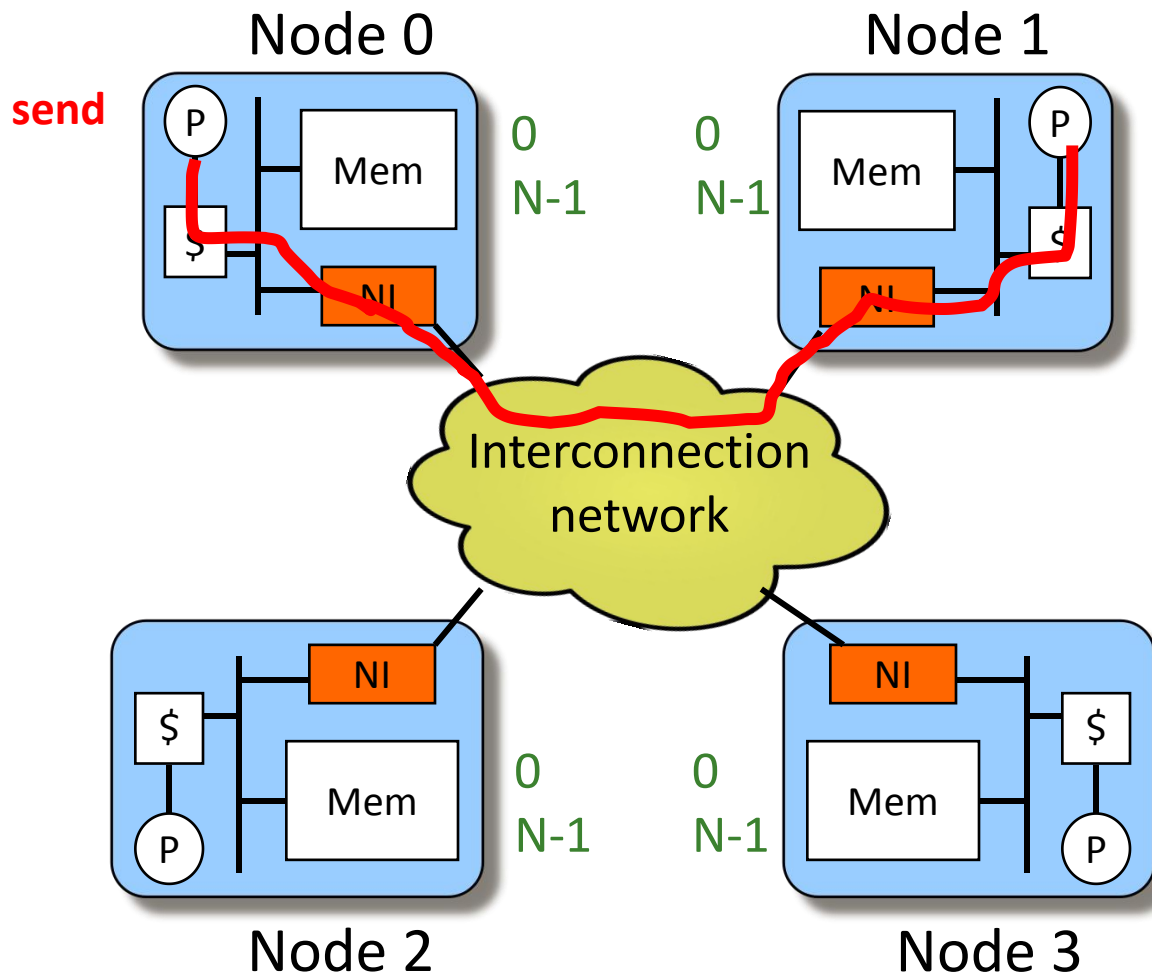
- 紧耦合多处理器: 10 个处理器

```
for (i = 1000*pid; i < 1000*(pid+1); i++)  
    sum = sum + A[i]; // critical section!
```

- 松耦合多处理器

```
for (i=0; i<1000; i++) sum = sum + A[i];  
if (pid != 0) send(0,sum);  
else for(i=1; i<9; i++) {  
    receive(i,partial_sum);  
    sum = sum + partial_sum;}  
}
```

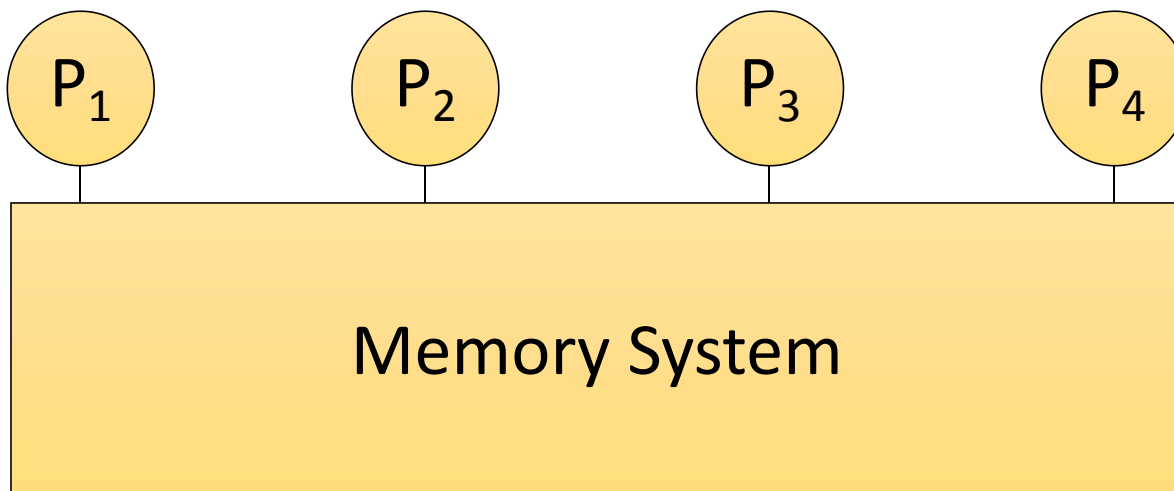
松耦合多处理器



- 每个节点有私有内存
- 不能直接访问其他节点的内存
- 使用send/recv函数交换数据
- 数据分配非常重要
- IBM SP-2, cluster of workstations
- MPI programming

紧耦合多处理器

- 多线程/处理器采用共享内存(地址空间)
- 通信是通过load和store隐式进行的



为何采用共享内存?

□ 优点:

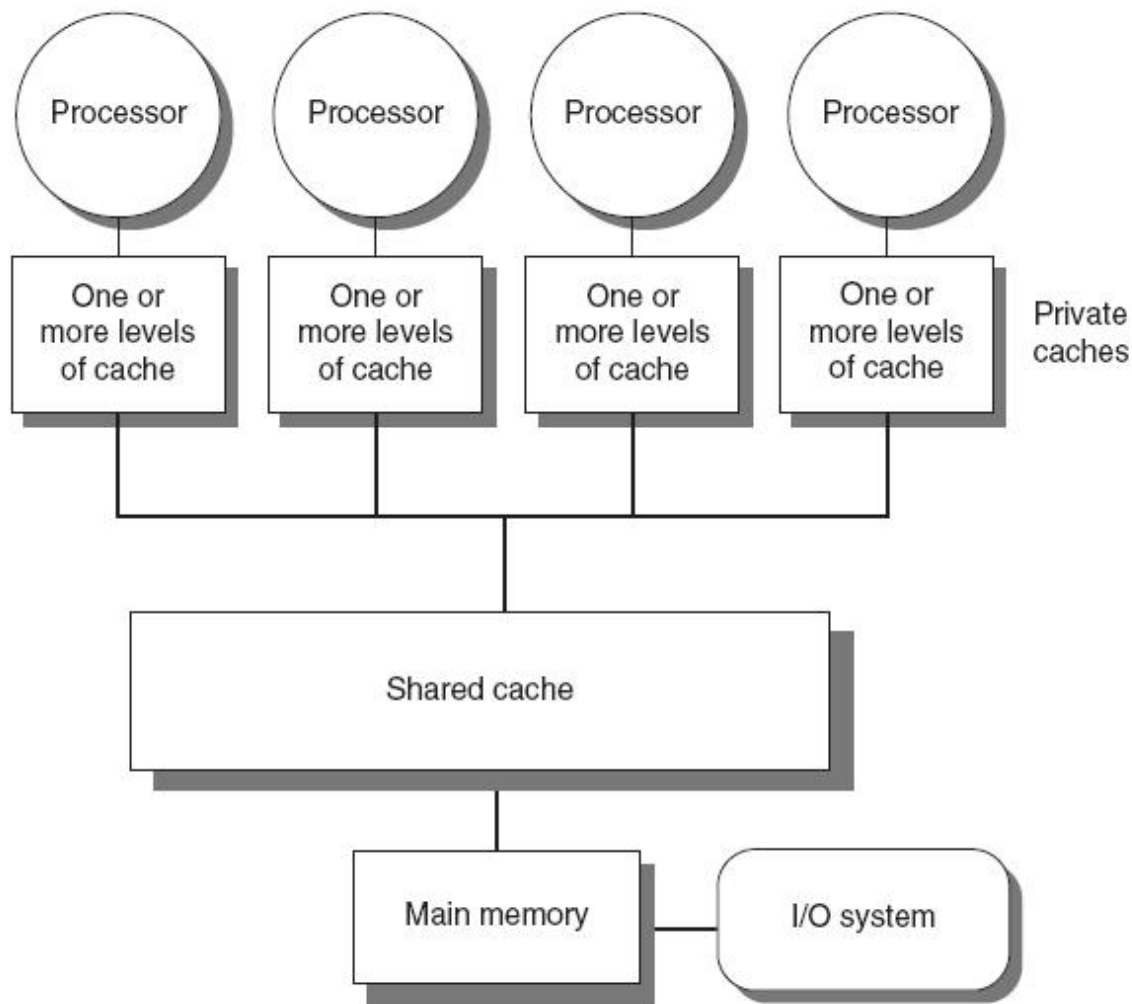
- 任务看到的是多任务单处理器
- 熟悉的编程模型, 类似多任务单处理器, 并且不需要管理数据分配
- OS 只需要渐进式扩展
- 不需要通过OS进行通信

□ 缺点:

- 同步策略复杂
- 通信是隐式的、间接的 (优化困难)
- 硬件实现困难

紧耦合多处理器: 两种类型

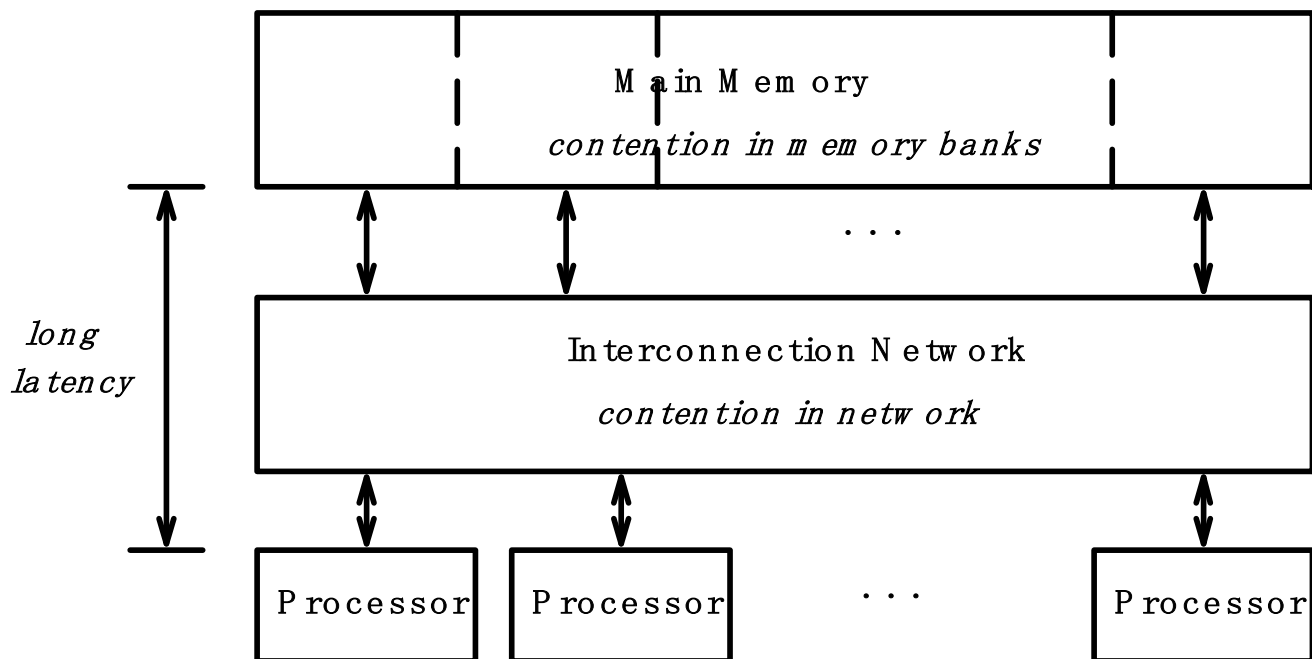
- 对称多处理器
(Symmetric multiprocessors, SMP)
 - 核心数量少
 - 共享单一内存，具有统一的内存访问延迟(UMA)



UMA: Uniform Memory/Cache Access

□ 所有核心具有相同的内存访问延迟

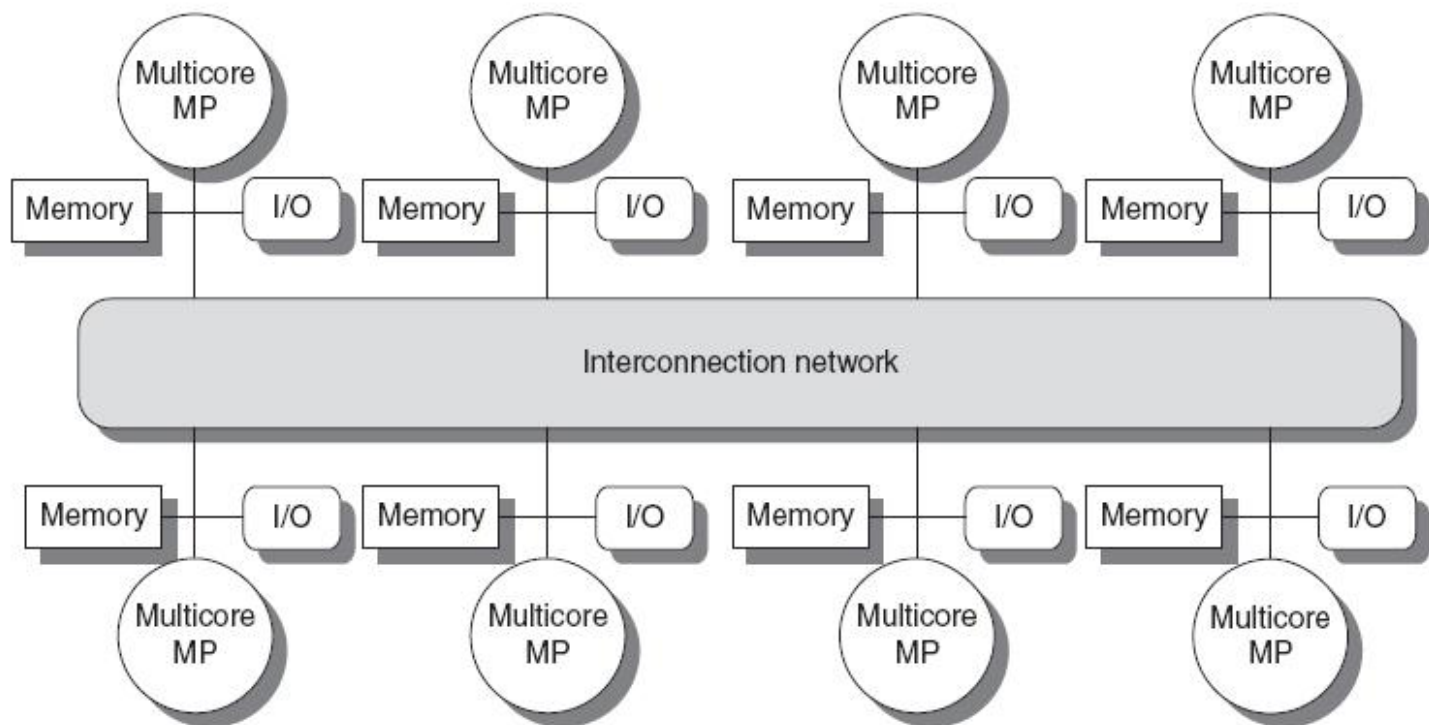
- 随着系统规模增长，延迟会增加
- + 数据放置策略不重要
- - 内存带宽竞争会增加访问延迟



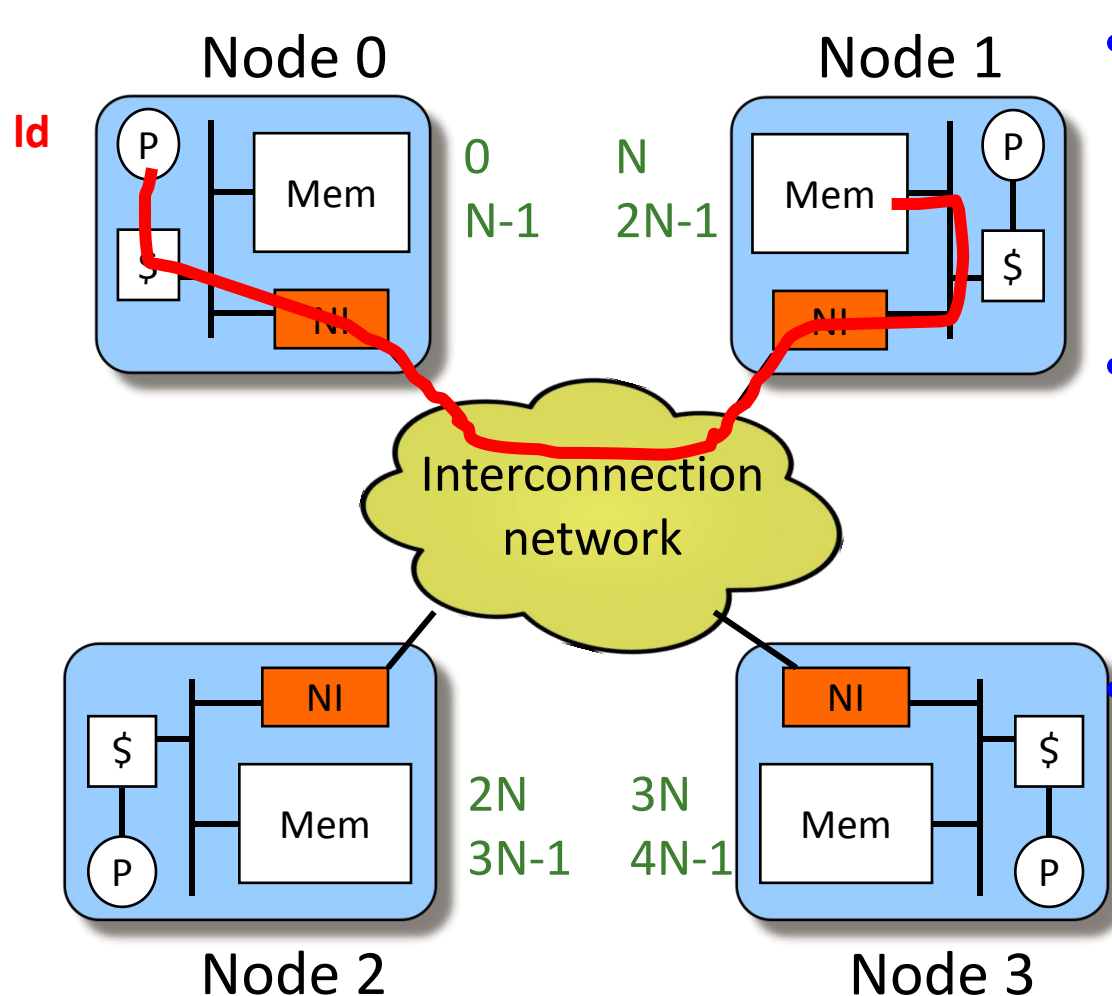
紧耦合多处理器: 两种类型

□ 分布式共享内存 (DSM)

- 内存分布在处理器之间 → 核心数量更多
- 非统一内存访问延迟(Non-uniform memory access/latency (NUMA))



分布式共享内存 (DSM)

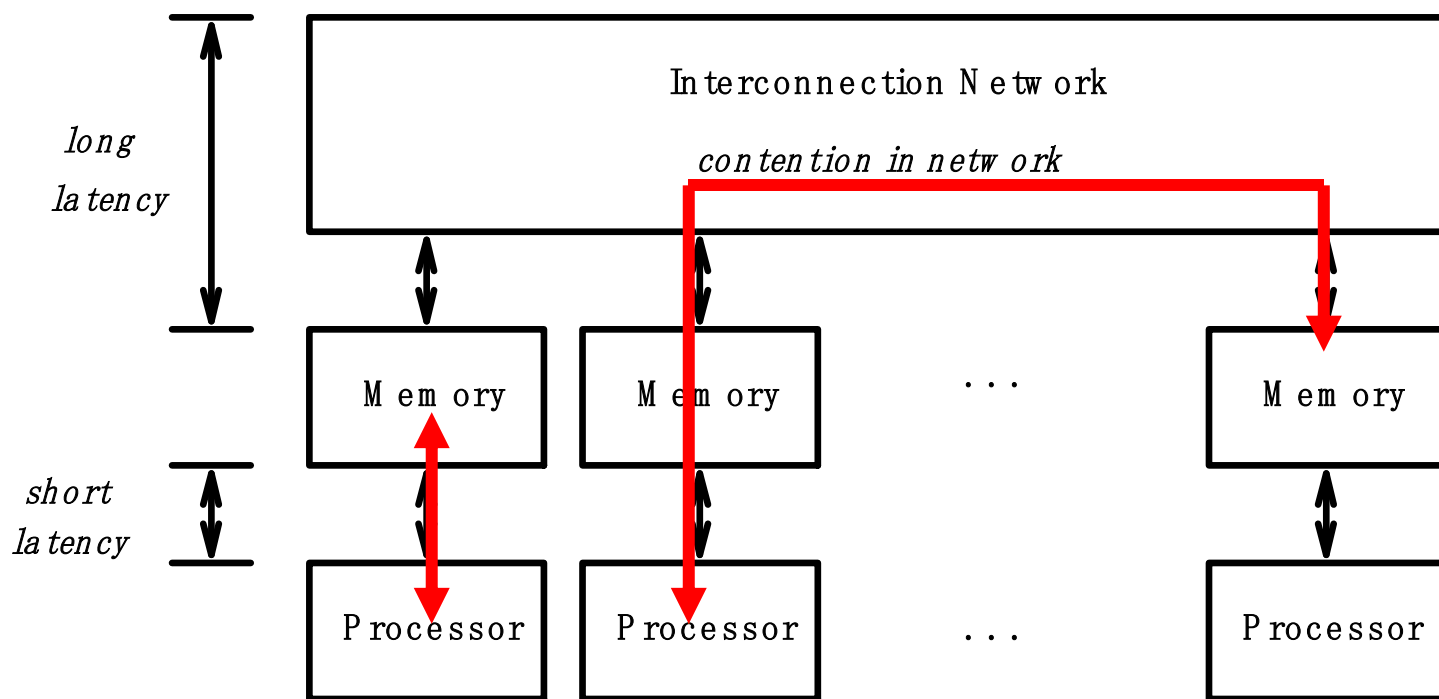


- 所有本地内存都通过一个全局地址空间进行寻址
- 一个节点可以直接访问其他节点内存，使用普通的load/store指令
- 节点通过直接（交换式）或多跳互连网络连接

NUMA: NonUniform Memory/Cache Access

□ 共享内存分为本地内存和远程内存

- + 本地内存访问低延迟, 高带宽
- - 远程内存访问延迟高很多
- - 性能对数据分布很敏感



并行性的注意事项

□ Amdahl' s Law

- f: 可并行部分占比
- N: 处理器个数

$$\text{Speedup} = \frac{1}{1 - f + \frac{f}{N}}$$

□ 最大加速比由串行部分的占比决定

□ 并行部分通常也不能实现完美并行

- 同步开销
- 负载分布不均
- 资源竞争

并行部分的瓶颈

□ 同步开销: 针对共享数据的操作无法并行化

- 锁, 互斥, barrier同步等
- 线程之间通信
- 当访问共享数据发生竞争时, 会导致线程的串行化

□ 负载不均: 并行任务执行时间不等

- 并行化不完美或者微体系结构影响
- 会降低加速比

□ 资源竞争: 并行任务共享资源, 产生竞争

- 增加共享资源成本较高
- 任务单独运行时不会产生问题

紧耦合多处理器存在的问题

- 如何发掘程序的并行性
- 远程访问延迟高
- 共享内存同步开销
 - Locks, atomic operations
- Cache一致性和内存一致性
- 内存访问次序问题
- 资源共享，竞争，划分问题
- 通信: 互联网络
- 负载不均衡

本课程讨论的多处理器主要是紧耦合多处理器